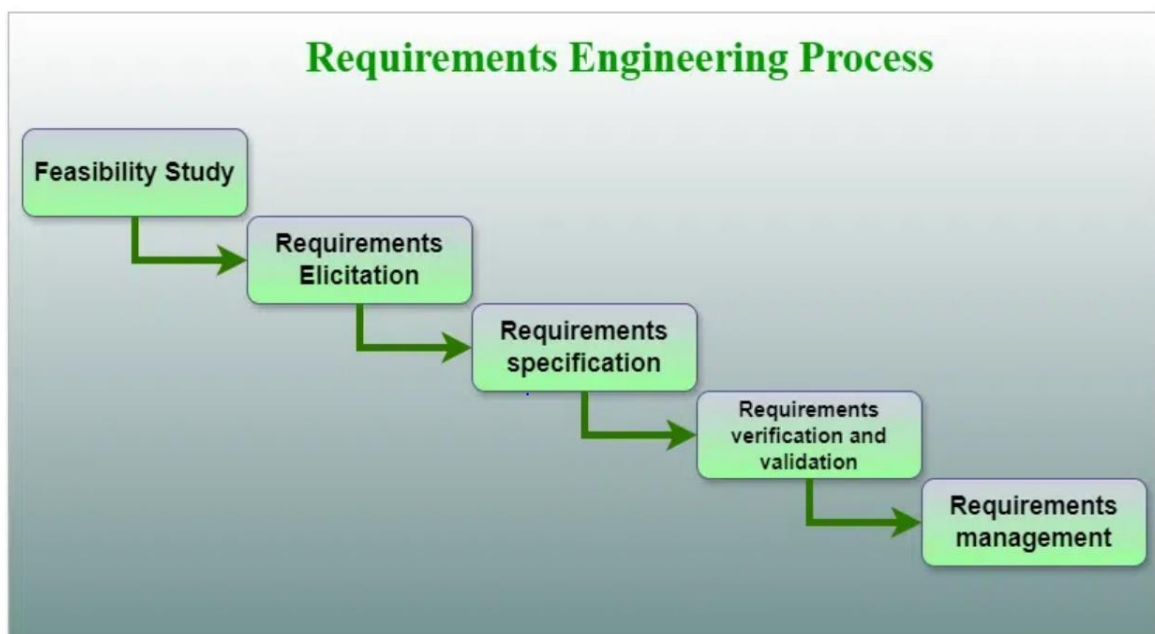


## Chapter 3: Analysis Concepts and principles

### ❖ Requirement Engineering:

- **Requirements Engineering (RE)** is the process of identifying, eliciting, analyzing, specifying, validating, and managing the needs and expectations of stakeholders for a software system. Requirement engineering provides the appropriate mechanism to understand what the customer desires, analyzing the need, and assessing feasibility, negotiating a reasonable solution, specifying the solution clearly, validating the specifications and managing the requirements as they are transformed into a working system. Thus, requirement engineering is the disciplined application of proven principles, methods, tools, and notation to describe a proposed system's intended behavior and its associated constraints.

### **Requirements Engineering Process:**



1. Feasibility Study
2. Requirements Elicitation
3. Requirements specification
4. Requirements for verification and validation
5. Requirements management

## 1. Feasibility Study:

A **feasibility study** in Requirements Engineering is simply an early, detailed check to see if a proposed software project is actually **possible and worthwhile** before any major development work begins. The feasibility study mainly concentrates on below five mentioned areas below.

- **Technical Feasibility:** In Technical Feasibility current resources both hardware software along required technology is analyzed/assessed to develop the project. This technical feasibility study reports whether there are correct required resources and technologies that will be used for project development.
- **Operational Feasibility:** In Operational Feasibility degree of providing service to requirements is analyzed along with how easy the product will be to operate and maintain after deployment.
- **Economic Feasibility:** Economic feasibility decides whether the necessary software can generate financial profits for an organization.
- **Legal Feasibility:** In legal feasibility, the project is ensured to comply with all relevant laws, regulations, and standards. It identifies any legal constraints that could impact the project and reviews existing contracts and agreements to assess their effect on the project's execution.
- **Schedule Feasibility:** In schedule feasibility, the project timeline is evaluated to determine if it is realistic and achievable. Significant milestones are identified, and deadlines are established to track progress effectively.

## 2. Requirements Elicitation and analysis:

This phase involves gathering and analyzing requirements from stakeholders including users, customers, and domain experts. The goal is to understand the needs of the system and translate them into specific software requirements. This process involves several techniques, such as:

- **Interviews:** Direct communication with stakeholders to gather information.
- **Surveys:** Distributing questionnaires to collect opinions and preferences.

- **Workshops:** Group sessions to brainstorm and gather requirements collaboratively.
- **Observation:** Actively observing users to understand their workflow and needs.
- **Prototyping:** Building a preliminary version of the software to elicit feedback.

### 3. Requirements Specification:

In this phase, the gathered and analyzed requirements are documented formally. Requirements specification is the process of documenting the requirements identified in the analysis step in a clear, consistent, and unambiguous manner. This step also involves prioritizing and grouping the requirements into manageable chunks. The goal of this step is to create a clear and comprehensive document that describes the requirements for the software system. The SRS includes the following components:

- **Functional Requirements:** Descriptions of what the system should do in terms of specific functions and features.
- **Non-functional Requirements:** Constraints and qualities the system must possess (e.g., performance, security).
- **User Interfaces:** Descriptions of how users will interact with the system.
- **Data Requirements:** Details about the data the system will store, process, and manage.

### 4. Requirements verification and validation:

Verification is checking that the requirements are complete, consistent, and accurate. It involves reviewing the requirements to ensure that they are clear, testable, and free of errors and inconsistencies. This can include reviewing the requirements document, models, and diagrams, and holding meetings and walkthroughs with stakeholders.

Validation is the process of checking that the requirements meet the needs and expectations of the stakeholders. It involves testing the requirements to ensure that they are valid and that the software system being developed will meet the needs of the stakeholders.

## **5. Requirements Management:**

Requirements management is the process of managing the requirements throughout the software development life cycle, including tracking and controlling changes, and ensuring that the requirements are still valid and relevant.

It includes activities such as:

- **Version Control:** Managing different versions of the requirements document.
- **Change Control:** Managing and documenting changes to requirements and their impact.
- **Traceability:** Establishing links between requirements and design, implementation, and testing.
- **Prioritization:** Assigning priorities to requirements to guide development efforts.
- **Communication:** Ensuring stakeholders are informed about changes and updates to requirements.

## **Advantages of Requirements Engineering Process**

- Helps ensure that the software being developed meets the needs and expectations of the stakeholders
- Can help identify potential issues or problems early in the development process, allowing for adjustments to be made before significant
- Helps ensure that the software is developed in a cost-effective and efficient manner
- Can improve communication and collaboration between the development team and stakeholders
- Provides an unambiguous description of the requirements, which helps to reduce misunderstandings and errors.
- Helps to ensure that the software system is delivered on time, within budget, and to the required quality standards.
- Provides a solid foundation for the development process, which helps to reduce the risk of failure.

## Disadvantages of Requirements Engineering Process

- Can be time-consuming and costly, particularly if the requirements-gathering process is not well-managed
- Can be difficult to ensure that all stakeholders' needs and expectations are taken into account
- It Can be challenging to ensure that the requirements are clear, consistent, and complete
- Changes in requirements can lead to delays and increased costs in the development process.
- It can be time-consuming and expensive, especially if the requirements are complex.
- There may be conflicts between stakeholders, which can be difficult to resolve.
- It may be challenging to ensure that all stakeholders understand and agree on the requirements.

### **3.1 Requirement Elicitation:**

Requirements elicitation is the process of gathering and defining the requirements from all relevant stakeholders for a software system. The goal of requirements elicitation is to ensure that the software development process is based on a clear and comprehensive understanding of the customer's needs and requirements.

#### **Importance of Requirement Elicitation**

- **Aligns with Business Goals:** Ensures the system matches the company's objectives and delivers real value.
- **Improves User Satisfaction:** Involving users helps build a system that meets their needs and expectations.
- **Saves Time and Cost:** Clear requirements reduce misunderstandings, rework, and delays.

- **Ensures Compliance:** Helps identify legal, regulatory, and industry standards that the system must follow.
- **Supports Traceability:** Well-documented requirements make it easy to track, test, validate, and maintain the system.

## **Techniques for Requirement Elicitation:**

### **1. Initiating the Process**

Initiating the Process is the first step in requirement elicitation where the development team prepares the foundation for collecting requirements. It ensures everyone understands the problem, objectives, and roles before gathering detailed requirements.

- Focus: High-level understanding and preparation.
- Activities:
  - Identify stakeholders (e.g., clients, end-users, team members).
  - Define the overall goal of the system or project.
  - Set expectations, roles, and responsibilities.
  - Ensure stakeholders are aligned on objectives and the process moving forward.

A clear roadmap or plan for how the requirement elicitation will be conducted.

### **2. FAST (Facilitated Application Specification Techniques)**

Facilitated Application Specification Techniques are team-based methods used to collect and define the requirements of a software system. Instead of a single analyst preparing requirements, these techniques involve group discussions, workshops, and structured meetings to gather correct, complete, and consistent requirements from all stakeholders.

Its objective is to bridge the expectation gap - the difference between what the developers think they are supposed to build and what customers think they are going to get. A team-oriented approach is developed for requirements gathering.

### **Benefits:**

- Improves communication and collaboration between customers and developers.
- Helps to better understand customer needs and project objectives.
- Can lead to a higher quality of requirements and help avoid cost overruns by detecting errors early.

### **3. Quality Function Deployment (QFD)**

- Quality Function Deployment (QFD) is process or set of tools used to define the customer requirements for product and convert those requirements into engineering specifications and plans such that the customer requirements for that product are satisfied.
- QFD aims at translating Voice of Customer into measurable and detailed design targets and then drives them from the assembly level down through sub-assembly level, component level, and production process levels.
- QFD helps to achieve structured planning of product by enabling development team to clearly specify customer needs and expectations of product and then evaluate each part of product systematically.

### **Challenges in Requirements Elicitation:**

- **Unclear or Vague Requirements**  
Stakeholders may not know exactly what they want or may express their needs ambiguously, making them difficult to understand.
- **Communication Barriers**  
Misunderstandings can occur due to poor communication between stakeholders and analysts. Cultural, language, or domain knowledge gaps can add to the difficulty.
- **Lack of Stakeholder Engagement**  
Some stakeholders may be uninterested, busy, or unwilling to participate fully.
- **Conflicting Requirements**  
Different stakeholders may have opposing needs or priorities, leading to conflicts.

- **Changing Requirements**

Requirements can evolve due to changing business needs, regulations, or market conditions.

- **Managing Expectations**

Stakeholders may have unrealistic expectations regarding system capabilities, time, or budget.

- **Resistance to Change**

Some stakeholders may resist new processes or system features and may not provide input or may reject proposed changes.

### **3.2 Use Cases and User stories:**

- ❖ **Use cases:**

- A Use Case describes how an actor (a user or external system) interacts with the system to achieve a specific, valuable goal. It focuses on the functionality of the system and the flow of events.

#### **Purpose of Use Cases:**

- **Capture functional requirements clearly**

- Use cases describe **what the system should do** in a structured way.
- Example: In an ATM system, a use case like **Withdraw Cash** clearly defines that the system should allow a customer to withdraw money.
- Benefit: Developers know exactly **what functionality** to implement.

- **Show how a system interacts with users**

- Use cases **highlight the interaction** between users (actors) and the system.
- Example: In online shopping, a **Place Order** use case shows the steps: user selects items → adds to cart → makes payment → receives confirmation.
- Benefit: Stakeholders can **see the flow** of system usage.

- **Serve as a communication tool between stakeholders and developers**



- Use cases provide a **common language** for both technical and non-technical people.
- Example: A manager can understand the **Login** use case without needing to know the code behind it.
- Benefit: Avoids misunderstandings about system requirements.

➤ **Can be used for testing and validation**

- Testers can create **test cases** based on use case steps.
- Example: For **Withdraw Cash**, test cases can check if the ATM correctly verifies PIN, dispenses cash, and updates account balance.
- Benefit: Ensures system works as expected and meets requirements.

❖ **User Stories:**

- A user story is a short, simple description of a user's need written from the user's perspective.
- User stories are used to capture requirements in an agile project and help the development team understand the needs and expectations of the users.

**INVEST Principle of User story :**

A good user story should be based on INVEST principle which expresses the quality of the user story because in base a good software product is completely dependent upon a good user story. In 2003 INVEST checklist was introduced by Bill Wake in an article.

- **Independent:**  
A user story should be **self-contained** and not depend heavily on other stories. This makes planning, development, and prioritization easier.
- **Negotiable:**  
A user story should not be written as a fixed contract.  
It should remain **flexible** so that details can be discussed and adjusted between the team and stakeholders.
- **Valuable:**  
A user story must clearly **deliver value to the user or customer**. If it doesn't provide value, it should not exist.

- **Estimable:**  
The team should be able to **estimate the size, effort, or time** required to complete the story. If it's too vague, it cannot be estimated.
- **Small:**  
A user story should be **small, simple, and manageable**. Large or complex stories should be broken down into smaller ones.
- **Testable:**  
A user story must include **clear acceptance criteria** so that it can be tested and verified after implementation. If you cannot test it, you cannot call it "done".

### 3.3 Requirement Specification:

- Requirements specification is the process of documenting the requirements identified in the analysis step in a clear, consistent, and unambiguous manner, so that developers, testers, and stakeholders understand what the system should do. The goal of this step is to create a clear and comprehensive document that describes the requirements for the software system. The final document is often called **Software Requirements Specification (SRS)**.

#### Essential Components of SRS

- Introduction
- Overall Description/ System features
- Specific Requirements (Functional & Non-Functional)
- External Interface Requirements

### Let's take an example of School Management System Project

#### 1. Introduction

##### 1.1 Purpose

School Management System (SMS) is to streamline and automate the administrative and academic processes of a school, improving efficiency, communication, and access to information for all stakeholders, including administrators, teachers, students, and parents.

##### 1.2 Scope

The SMS is a web-based system designed to automate school operations such as admissions, attendance tracking, examination management, and reporting. It provides interfaces for students, teachers, and administrators.

### **1.3 Definitions, Acronyms, and Abbreviations**

- SMS: School Management System
- Admin: School Administrator
- DB: Database

### **1.4 References**

- IEEE SRS Standard Template
- School Administration Policies

### **1.5 Overview**

SMS is a software solution designed to streamline and automate various school operations, including student admissions, attendance tracking, timetable management, exam scheduling, and report generation. It enhances efficiency, fosters better communication between stakeholders, and provides easy access to information for administrators, teachers, students, and parents, creating a more organized and effective educational environment.

## **2. Overall Description/ System features**

### **2.1 System Context**

- The SMS will integrate with existing school databases and can be accessed via web browsers. The key stakeholders include administrators, teachers, students, and parents.

### **2.2 Product Features**

- Student Management: Admission, profile, grades, and attendance.
- Teacher Management: Profile, class schedules, and performance tracking.
- Attendance Tracking: Digital attendance system for students and teachers.

- Examination Module: Schedule, conduct, and manage results.
- Reports: Generate performance and attendance reports.

### **2.3 Assumptions and Dependencies**

- Teachers and students will have basic computer literacy.
- The school has a stable internet connection for system access.

## **3. Functional Requirements**

### **3.1 Student Management**

- Add, update, and delete student records.
- View student grades and attendance.

### **3.2 Teacher Management**

- Assign classes and schedules.
- View teacher performance reports.

### **3.3 Attendance Tracking**

- Digital marking of attendance via an interface.
- Export attendance data.

### **3.4 Examination Management**

- Create examination schedules.
- Record and view results.

### **3.5 Reporting**

- Generate detailed reports on performance and attendance.
- Export reports in various formats (PDF, Excel).

## **4. Non-Functional Requirements**

- Scalability: Support up to 10,000 users.
- Security: Role-based access control and data encryption.

- Usability: Simple and intuitive user interface.
- Performance: The system should load within 3 seconds under normal conditions.
- All reports must be generated within 5 seconds.

## 5. Technical Requirements

- Development Model: Agile development model
- Frontend: Angular
- Backend: .NET Core 8
- Database: SQL Server
- Hosting: Cloud-based deployment

### 3.4 Analysis Principles:

- Analysis is the process of studying what the system should do before designing or implementing it. It focuses on understanding requirements and modeling the system properly.
- Analysis Principles are guidelines to ensure proper and effective analysis of a system.

#### Key Principles of Analysis Modelling

1. **Abstraction:** Analysis modelling involves separating important system components from unneeded specifics. While leaving out unnecessary or low-level information, it concentrates on capturing the essential ideas, behaviors, and relationships relevant to the system's requirements.
2. **Modularity:** Analysis models ought to be able to break down a system into smaller, more manageable parts. It is simpler to understand, assess, and alter the system when each module or component reflects a different part of its functionality.
3. **Consistency:** Internally and with other project artifacts, including requirements documents, design specifications, and implementation code, analysis models should be consistent. By preventing opposing or

conflicting representations of the system, consistency promotes greater stakeholder comprehension and alignment.

4. **Traceability:** Analysis models ought to be able to be linked to other project components so that interested parties may follow requirements from their inception to their execution. Throughout the software development lifecycle, it helps with impact analysis, change management, and requirements coverage verification.
5. **Precision:** To provide an unambiguous picture of the needs and behaviors of the system, analysis models must be accurate and exact. Accuracy lowers the chance of miscommunication and misunderstanding among stakeholders as well as implementation problems.
6. **Separation of Concerns:** Analysis modeling divides various system components or concerns into discrete representations. For instance, behavioral modeling aims to capture the dynamic behavior of the system, whereas data modeling concentrates on expressing the relationships and structure of data items.

### **3.5 Software Prototyping, Specification, Specification Review**

#### **❖ Software Prototyping:**

- Software Prototyping is the process of creating an early, simplified version of a software application to visualize and test its core features, design, and functionality before full-scale development. It is used to gather feedback, validate requirements, and identify potential issues early in the development process.

#### **Explanation of Software Prototyping with a Real-Life Example**

##### **Scenario: Developing a Food Delivery App**

Let's imagine you're building a food delivery app like Foodmandu. Here's how software prototyping would work in this context:

##### **Step 1: Identify Core Features**

Before starting development, the team identifies the app's core features, such as:

- User registration and login.
- Restaurant browsing.
- Food ordering and payment.
- Delivery tracking.

##### **Step 2: Create a Prototype**

The team develops a simple prototype to demonstrate the basic functionality and design.

#### **Example: Low-Fidelity Prototype**

- A wireframe or sketch of the app is created using tools like Figma or Adobe XD.
- The prototype shows basic screens like the login page, restaurant list, menu, and cart.
- There's no backend or working features, just static designs to visualize user flow.

#### **Example: High-Fidelity Prototype**

- The prototype includes clickable buttons and transitions to simulate user interaction.
- Clicking on a restaurant opens the menu.
- Adding food to the cart updates the cart page.

#### **Step 3: Test and Gather Feedback**

The prototype is shared with key stakeholders (e.g., restaurant owners, delivery partners, and potential users).

#### **User Feedback Example:**

- Testers might say, "The cart icon isn't visible enough" or "Adding a favourite restaurant option would be useful."
- The feedback helps refine the app's usability and features.

#### **Step 4: Iterate and Improve**

The team iterates on the prototype based on feedback:

- Adding a visible cart button.
- Including a "favourites" section for users.

#### **Step 5: Build the Final Product**

Once the prototype aligns with user expectations, the development team builds the final app, incorporating the refined features and workflows.

#### **❖ Software Specification:**

- Software specification is the process of defining and documenting the requirements, features, constraints, and goals of a software system in a detailed and structured manner. It ensures that both developers and stakeholders have a clear understanding of what the software must achieve.

#### **Key Elements of Software Specification**

★ **Functional Requirements:**

- Describes what the system should do.
- Example: "The system shall allow users to log in with their email and password."

★ **Non-Functional Requirements:**

- Defines the quality attributes such as performance, scalability, and security.
- Example: "The system shall handle 1,000 concurrent users with response times under 2 seconds."

★ **Interface Requirements:**

- Details interactions with external systems or components.
- Example: "The system shall integrate with a third-party payment gateway."

★ **Constraints:**

- Specifies limitations, such as technology choices, budget, or regulatory compliance.
- Example: "The application must comply with GDPR (General Data Protection Regulation)."

★ **Assumptions and Dependencies:**

- Highlights conditions assumed to be true and external factors affecting the project.
- Example: "The software will run on a cloud-hosted infrastructure."

❖ **Software Specification review:**

- The Specification Review (often called **Software Specification Review** or SSR) is a formal, systematic examination of the requirements documentation for a software system, most commonly the Software Requirements Specification (SRS) document.
- It is a critical step in the early phases of the Software Development Life Cycle (SDLC) to ensure that the development team has a complete, clear, and accurate understanding of *what* the customer or stakeholders require before design and coding begin.
- The core purpose of a Specification Review is to **validate the requirements** themselves, not the code or the design.



A successful SSR aims to verify that the requirements possess the following key qualities:

- **Correct:** Each requirement accurately reflects the actual need.
- **Unambiguous:** Each requirement has only one interpretation.
- **Complete:** All necessary functional and non-functional requirements are present.
- **Consistent:** No requirement conflicts with another.
- **Verifiable (Testable):** There is a cost-effective process (like a test case) to prove the requirement has been met.
- **Traceable:** The origin of each requirement is clear, and it can be tracked to higher-level system objectives.
- **Feasible:** The requirements can be realistically implemented within the project's technical, budget, and schedule constraints.

### **Benefits of Software Specification Review**

- Reduces cost and rework by catching defects early
- Improves quality and reliability
- Ensures better communication between stakeholders
- Prevents misunderstandings during design and development
- Ensures the final product meets user requirements.

### **❖ Software Review:**

Software Review is a systematic inspection of software by one or more individuals who work together to find and resolve errors and defects in the software during the early stages of the Software Development Life Cycle (SDLC). A software review is an essential part of the Software Development Life Cycle (SDLC) that helps software engineers in validating the quality, functionality, and other vital features and components of the software. It is a whole process that includes testing the software product and it makes sure that it meets the requirements stated by the client.

### **Objectives of Software Review**

The objective of the software review is:

1. To improve the productivity of the development team.
2. To make the testing process time and cost-effective.
3. To make the final software with fewer defects.
4. To eliminate the inadequacies.

### Importance of Software Reviews:

- Detect defects early, reducing cost and effort.
- Improve the quality and reliability of software.
- Ensure alignment with standards and requirements.
- Foster collaboration and knowledge sharing among team members.

### 3.6 Analysis Modeling

- Analysis modeling in software engineering is the process of creating a technical representation of a system's information, functions, and behavior. It helps in understanding customer requirements, provides a foundation for software design, and defines a set of requirements that can be reviewed and validated.

#### Element of Analysis Modeling:

- I. Data Modeling
- II. Functional Modeling
- III. Behavioral Modeling
- IV. Structural Modeling

#### I. Data modelling (How data is structured and stored)

- Data modeling is the process of creating a simplified visual diagram of the data elements a system will store and how they are logically related.
- Focuses on structuring data, relationships, and attributes.
- Used to design the database part of the system.
- **Tools: Entity-Relationship Diagrams (ERD), Data Dictionary.**

#### Importance of data modeling:

- **Clarity & Consistency:** Provides a clear structure of entities, attributes, and relationships, ensuring consistent data understanding.
- **Efficiency:** Improves data storage, retrieval, and overall system performance.
- **Scalability:** Supports future growth by handling increasing data without reducing performance.
- **Data Integrity:** Ensures accuracy and reliability through validation rules and constraints.
- **Alignment with Business Needs:** Reflects business rules so the system meets organizational requirements.

- **Design Guidance:** Helps in designing efficient database schemas and application architecture.

### Real-world Examples of Data Modeling:

Real world applications of data modeling in system designing include:

- **E-commerce Platform:** Creating catalog models, which are used in customer profiles, orders, and transactions to provide for the online purchasing experience and inventory management.
- **Healthcare System:** Implementing record, medical history, and appointment design templates for the easy EHR Management and smooth healthcare workflows.

## II. Functional Modeling (What the system should do)

- It is the process of creating a model that defines the functions, services, or tasks that a system must perform. It is the action perspective of the system, focusing on what the system should do to meet user requirements, rather than how it will be built (design).
- **Tools: Use Case Model, DFD (Level 0/1 DFDs)**

### Functional modeling helps to:

- Identify major functions or services of the system.
- Understand how inputs are transformed into outputs.
- Show interactions between users and the system.
- Provide a clear picture of how data flows through the system.
- Form the foundation for system design, testing, and requirement validation.

### Real-Time Example: Online Shopping Website

- Search product
- Add to cart
- Make payment
- Place order

### III. Behavioral Modeling (How the system behaves or reacts)

- A behavioral model in software engineering represents how a system behaves and responds to events over time. It focuses on interactions, data flow, and state changes occurring during system execution. One of the key diagrams used to represent this is the State Transition Diagram (State Machine Diagram).
- It shows what happens or what should happen when the system reacts to **stimuli** (something that triggers the system to react) from its environment.
- There are mainly two types of stimuli in behavioral modeling:

**Data:** The system receives input data that must be processed.

**Events:** The system performs certain actions when a specific event occurs (e.g., button click, message arrival)

- **Tools: Sequence Diagram, Activity Diagram, State Machine Diagram.**

#### **Real-Time Example: ATM Machine**

Idle → Card Inserted → PIN Verified → Cash Dispensed → Idle

#### **Advantages:**

- Behavior and working of a system can easily be understood without any effort.
- Results are more accurate by using this model.
- This model requires less cost for development as cost of resources can be minimal.
- It focuses on behavior of a system rather than theories.

#### **Disadvantages:**

- This model does not have any theory, so trainee is not able to fully understand basic principle and major concept of modeling.
- This modeling cannot be fully automated.
- Sometimes, it's not easy to understand overall result.
- Does not achieve maximum productivity due to some technical issues or any errors.

#### IV. Structural Modeling (How the system is organized)

- Structural Modeling represents the static structure of a system—its components, objects, classes, and their relationships.
- It focuses on how the system is organized, including attributes, operations, and associations between elements.
- Shows the architecture of the system without describing behavior or processes.
- Used mainly in Object-Oriented Analysis and Design (OOAD).
- Tools: **Class Diagram, Object Diagram.**

##### Structural modeling helps to:

- Understand the architecture and framework of the system.
- Identify major components of the system.
- Define the relationships between objects or modules.
- Prepare a foundation for class design, database design, and coding.
- Provide a clear, static view of the system without showing execution behavior.

##### Real-Time Example: University Management System

Classes: Student, Course, Professor

Attributes: Student(Name, ID), Course(Code, Credits)

Relationships: Student enrolls in Course, Professor teaches Course

##### ❖ Information flow:

- Information flow is the movement of data through a system from input to processing to output, represented using diagrams like DFD to understand how information is generated, transformed, and stored in a system.
- Information flow is crucial in software engineering for aligning teams, reducing errors, ensuring quality, and enabling agility by ensuring clear, timely data movement between stakeholders (devs, clients, management) and system components, preventing miscommunication, aiding debugging (data flow testing), and

supporting secure, efficient design through structured diagrams like flowcharts and DFDs.

### **Importance of Information Flow:**

- **Collaboration & Alignment:**

Helps different teams (design, marketing, developers) work together with the same understanding so the work does not become disconnected.

- **Quality & Error Reduction:**

Finds mistakes early, improves testing, and ensures requirements are clear and correct.

- **Agility & Efficiency:**

Clear information flow helps teams work faster and adapt quickly. Regular updates and reports make development smoother.

- **Design & Architecture:**

Shows how the system works, how data moves, and how security is applied. Very helpful in complex systems.

- **Decision Making:**

Information moves upward as feedback for planning, and downward as instructions for work, helping management make better decisions.

### **❖ Structured analysis – data dictionary:**

- Structured Analysis is a systematic, top-down approach used to understand, analyze, and model system requirements before design and implementation with the help of tools like Data Flow Diagrams (DFD), structure charts, and data dictionaries.
- The **Data Dictionary** is a fundamental and essential component of Structured Analysis (SA).
- Data Dictionary is a central repository that stores the definitions and details of all data elements used in a system.
- It ensures that every term, data flow, and data store has a clear and consistent meaning.

## **Components of a Data Dictionary**

A Data Dictionary usually includes the following components:

### **1. Data Elements**

- The smallest units of data
- Example: Name, Age, Student\_ID

### **2. Data Structures**

- Combination of multiple data elements
- Example: Student\_Record = Student\_ID + Name + Age

### **3. Data Flows**

- Describes what data moves between processes in DFD
- Example: "Admission Form Data" flowing to the system

### **4. Data Stores**

- Places where data is stored
- Example: Student Database, File, Table

### **5. Data Types**

- Type of data such as integer, string, date

### **6. Data Format & Size**

- Length, pattern, and format of data
- Example: Phone = 10 digits, Date = DD/MM/YYYY

### **7. Allowed Values / Constraints**

- Rules or limits for data
- Example: Age must be between 1 and 100

### **8. Description / Meaning**

- Explanation of what the data represents.
- Example: Student\_ID = Unique number for each student.

### **9. Default Values (Optional)**

- Predefined values if user does not enter data.

## **10. Data Source & Destination (Optional)**

- Where data comes from and where it goes.

### **Benefits of Data Dictionary:**

- Ensures consistency and accuracy.
- Easy system maintenance.
- Better documentation and understanding.
- Reduces errors and misunderstandings.
- Supports design and implementation phases.

### **Role of Data Dictionary in Structured Analysis:**

- Works closely with Data Flow Diagrams.
- Defines all data flows clearly.
- Improves communication between users and developers.
- Helps break down complex systems clearly.
- Ensures correct and complete understanding of system data.

### **About some Diagram (imp for exam):**

#### **❖ ERD (Entity Relationship Diagram)**

- An ER Diagram is a visual representation of the data and the relationships among data in a system.
- It is used during database design to understand what entities (objects) exist, what attributes they have, and how they relate to each other.
- ER Diagrams are most often used to design or debug relational databases in the fields of software engineering, business information systems, education and research.



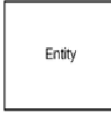
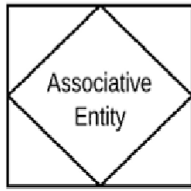
- ER Models use a defined set of symbols such as rectangles, diamonds, ovals and connecting lines to depict the interconnectedness of entities, relationships and their attributes.

### Components of ERD:

1. Entity
2. Attribute
3. Relationship

#### 1. Entity:

- Entities are objects or concepts that represent important data. Entities are typically nouns such as product, customer, location, or promotion.
- Represented by a **rectangle**.
- **Examples:** Student, Course, Department, Order, Product.

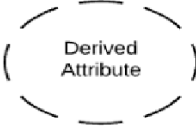
| Entity Name        | Symbol                                                                              | Meaning                                                                                                                                       |
|--------------------|-------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Strong entity      |  | Strong entities are independent from other entities, and are often called parent entities.                                                    |
| Weak Entity        |  | Weak entities depend on some other entity type. They don't have primary keys, and have no meaning in the diagram without their parent entity. |
| Associative Entity |  | Associative entities relate the instances of several entity types.                                                                            |

#### 2. Attribute:

- Attributes are the properties or characteristics of an entity. They are the columns in a table.
- Represented by an **oval** connected to the entity rectangle.

### Types of Attributes:

- **Simple Attribute:** Cannot be divided. (e.g., age)
- **Composite Attribute:** Can be divided. (e.g., full name → first name + last name)
- **Derived Attribute:** Value can be calculated. (e.g., age from DOB)
- **Multivalued Attribute:** Has multiple values. (e.g., phone numbers)

| Name                  | Symbol                                                                              | Description                                                                                   |
|-----------------------|-------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Attribute             |    | Attributes are characteristics of an entity                                                   |
| Multivalued attribute |   | Multivalued attributes are those that are can take on more than one value                     |
| Derived attribute     |  | Derived attributes are attributes whose value can be calculated from related attribute values |

### 3. Relationship:

- A relationship describes how two or more entities are associated with each other.
- Represented by a **diamond** connecting the participating entities.

### Types of Relationships:

#### 1. One-to-One (1:1)

- One entity of A relates to one entity of B.  
Example: Each **passport** is issued to only one **citizen**.

#### 2. One-to-Many (1:M)

- One entity of A relates to many of B.  
Example: A **teacher** teaches many **students**.

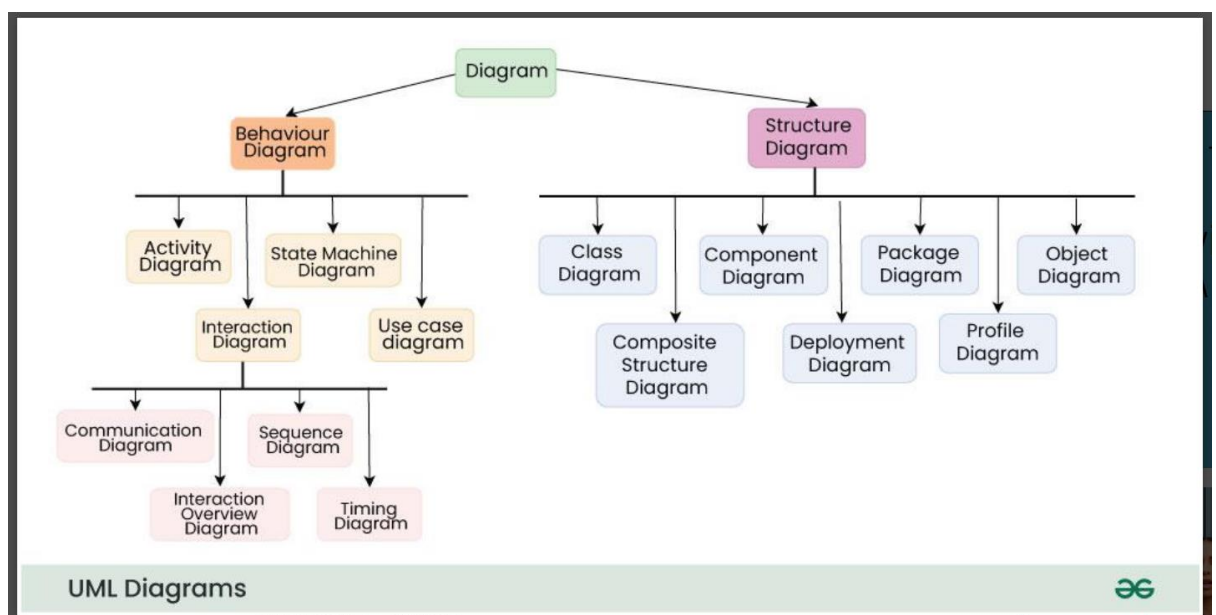
### 3. Many-to-Many (M:N)

- Many entities of A relate to many of B.  
Example: Students **enroll** in many courses.

## ❖ Use cases:

### UML

- Unified Modeling Language (UML) is a general-purpose modeling language. The main aim of UML is to define a standard way to visualize the way a system has been designed. It is quite similar to blueprints used in other fields of engineering. UML is not a programming language, it is rather a visual language.
- We use UML diagrams to show the behavior and structure of a system.
- UML helps software engineers, businessmen, and system architects with modeling, design, and analysis.

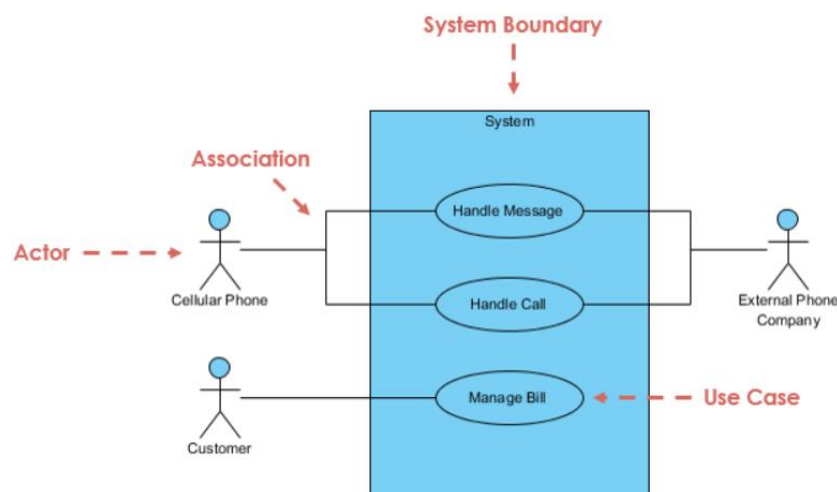


## Usecase Diagram

- Use case diagram models the behavior of the system.
- Use Case Diagrams are used to depict the functionality of a system or a part of a system. They are widely used to illustrate the functional requirements of the system and its interaction with external agents(actors).
- A use case is basically a diagram representing different scenarios where the system can be used.
- A use case diagram gives us a high level view of what the system or a part of the system does without going into implementation details. '

### Use Case Diagram Notations

UML notations provide a visual language that enables software developers, designers, and other stakeholders to communicate and document system designs, architectures, and behaviors in a consistent and understandable manner.



### 1. Actors

Actors are external entities that interact with the system. These can include users, other systems, or hardware devices. In the context of a Use Case Diagram, actors initiate use cases and receive the outcomes.

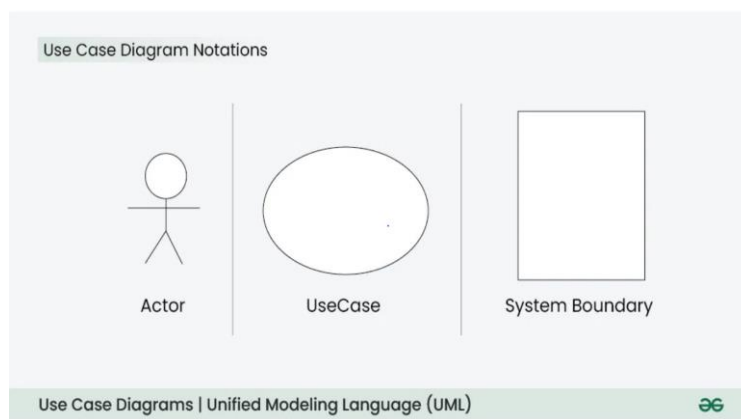
Proper identification and understanding of actors are crucial for accurately modeling system behavior.

## 2. Use Cases

Use cases are like scenes in the play. They represent specific things your system can do. In the online shopping system, examples of use cases could be "Place Order," "Track Delivery," or "Update Product Information". Use cases are represented by ovals.

## 3. System Boundary

The system boundary is a visual representation of the scope or limits of the system you are modeling. It defines what is inside the system and what is outside. System boundary represented by rectangle.

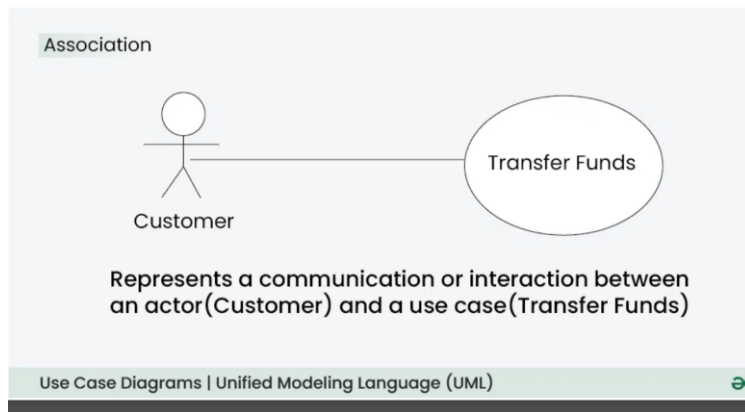


## Use Case Diagram Relationships

In a Use Case Diagram, relationships play a crucial role in depicting the interactions between actors and use cases.

### 1. Association Relationship

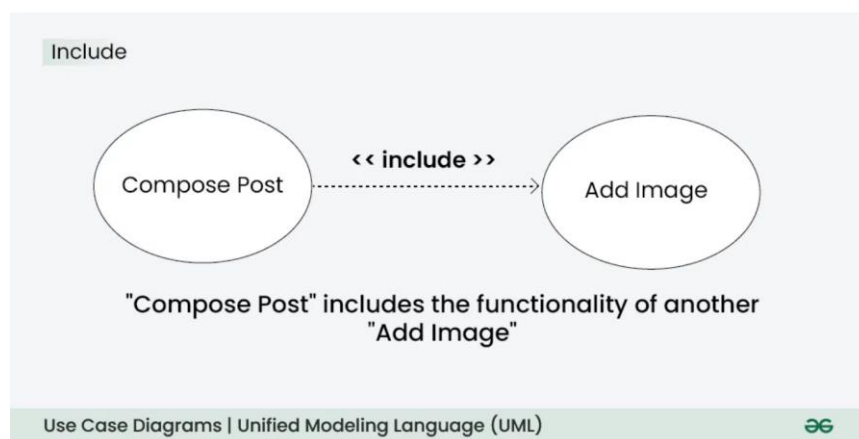
The Association Relationship represents a communication or interaction between an actor and a use case.



## 2. Include Relationship

The Include Relationship indicates that a use case includes the functionality of another use case. It is denoted by a dashed arrow pointing from the including use case to the included use case. This relationship promotes modular and reusable design.

- **Example: Social Media Posting**
- **Use Cases:** Compose Post, Add Image
- **Include Relationship:** The "Compose Post" use case includes the functionality of "Add Image." Therefore, composing a post includes the action of adding an image.

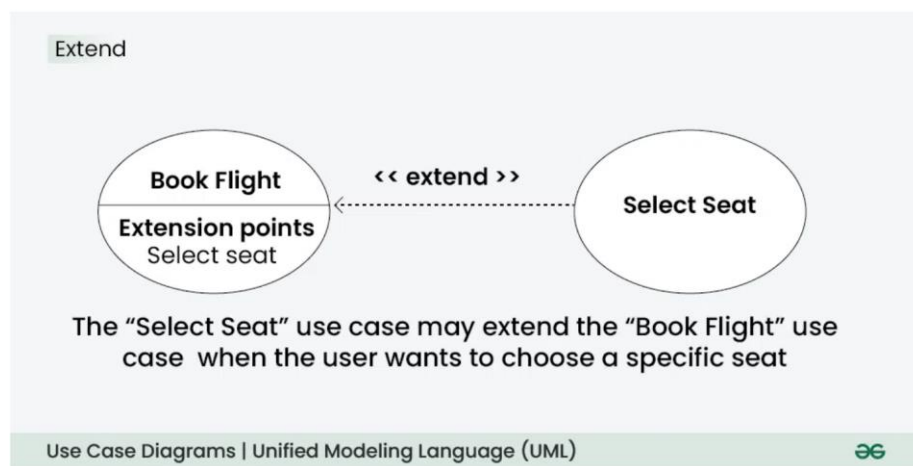


## 3. Extend Relationship

The Extend Relationship illustrates that a use case can be extended by another use case under specific conditions. It is represented by a dashed arrow with the

keyword "extend." This relationship is useful for handling optional or exceptional behavior.

- **Example: Flight Booking System**
- **Use Cases:** Book Flight, Select Seat
- **Extend Relationship:** The "Select Seat" use case may extend the "Book Flight" use case when the user wants to choose a specific seat, but it is an optional step.

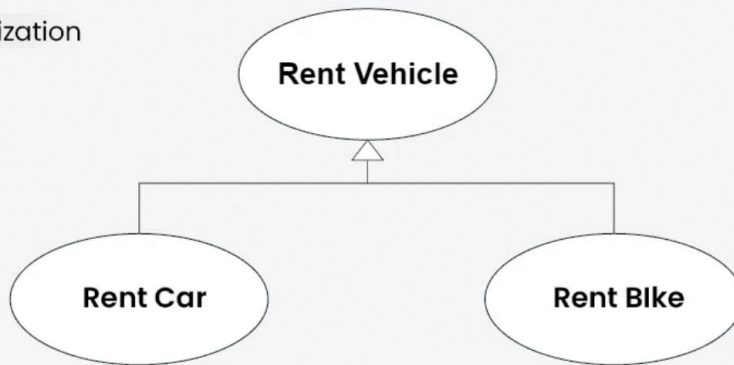


#### 4. Generalization Relationship

The Generalization Relationship establishes an "is-a" connection between two use cases, indicating that one use case is a specialized version of another. It is represented by an arrow pointing from the specialized use case to the general use case.

- **Example: Vehicle Rental System**
- **Use Cases:** Rent Car, Rent Bike
- **Generalization Relationship:** Both "Rent Car" and "Rent Bike" are specialized versions of the general use case "Rent Vehicle."

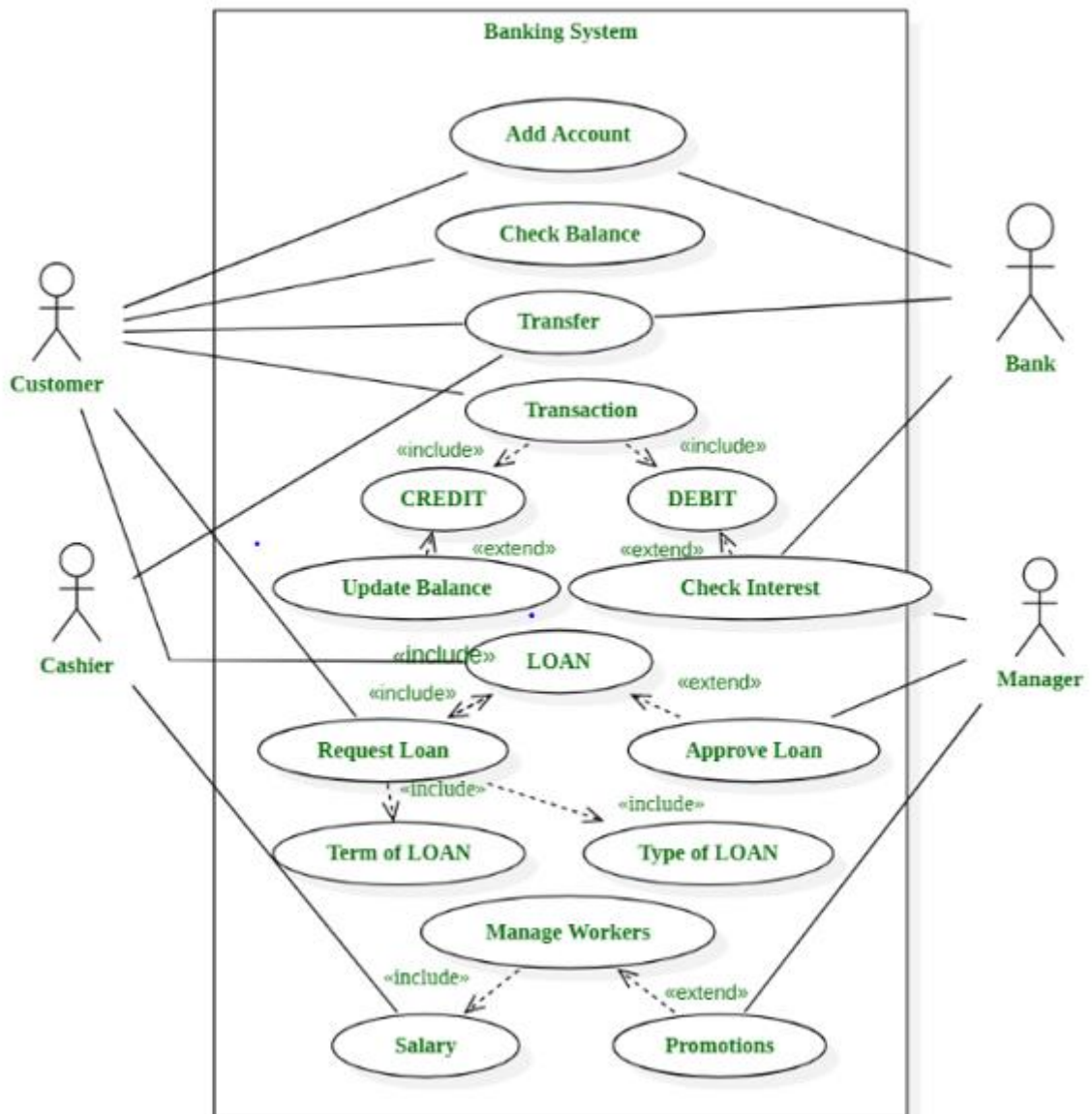
#### Generalization



Both "Rent Car" and "Rent Bike" are specialized versions of the general use case "Rent Vehicle."

**For example: online banking system**





### ❖ DFD (Data Flow Diagram):

- **A Data Flow Diagram (DFD)** is a graphical representation of the flow of data within any system.
- It shows how input data is transformed into output through various processes and helps identify the data sources, data storage, and data destinations in the system.
- DFDs can effectively illustrate the incoming, outgoing, and stored data in a system.

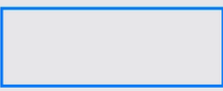
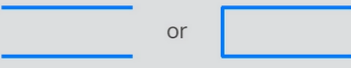
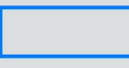
### Levels of Data Flow Diagrams (DFD):

DFDs are categorized into various levels, with each level providing different degrees of detail. The levels are numbered from **0** and onward. The higher the level, the more detailed the diagram becomes.

- **Level 0 DFD** (Context Diagram): Represents the entire system as one single process, showing its interaction with external entities. It highlights major inputs and outputs without internal details.
- **Level 1 DFD**: Breaks the Level 0 process into major subprocesses. Shows internal data flows and data stores.
- **Level 2 and Beyond**: Further decomposes Level 1 subprocesses for a detailed view of specific functional areas. Useful for large or complex systems requiring deeper analysis.

| Sr.No. | DFD Level | Purpose Detail             | Level                | Typical Components                                         |
|--------|-----------|----------------------------|----------------------|------------------------------------------------------------|
| 1      | Level 0   | Broad overview (context)   | High-level           | Single process, external entities, data flows              |
| 2      | Level 1   | Main system decomposition  | Medium               | Multiple processes, data stores, specific data flows       |
| 3      | Level 2   | Detailed process breakdown | Low-level (granular) | Sub-processes, additional data stores, detailed data flows |

## Symbol Used in DFD

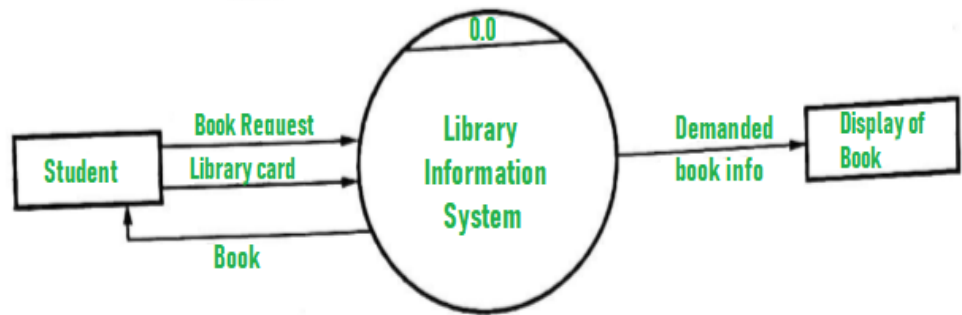
| Element                          | Description                                                                            | Symbol                                                                                                                                                                     |
|----------------------------------|----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Business Process                 | The activities within a system that use or generate data                               |                                                                                         |
| Entity or Data Source / Hardware | The entities that interact with the system (customers, employees, vendors, technology) |                                                                                         |
| Data Store or Data File          | Physical and electronic data storage                                                   |  or  |
| Data Flow                        | The flow of data identified with a directional arrow                                   |                                                                                         |

1. **Process:** Represents a function or activity that transforms data.
2. **Data flow:** Shows movement of data between processes, data stores, and external entities.
3. **Data store:** Stores data temporarily or permanently for later use.
4. **External entity:** Represents a source or destination of data outside the system.

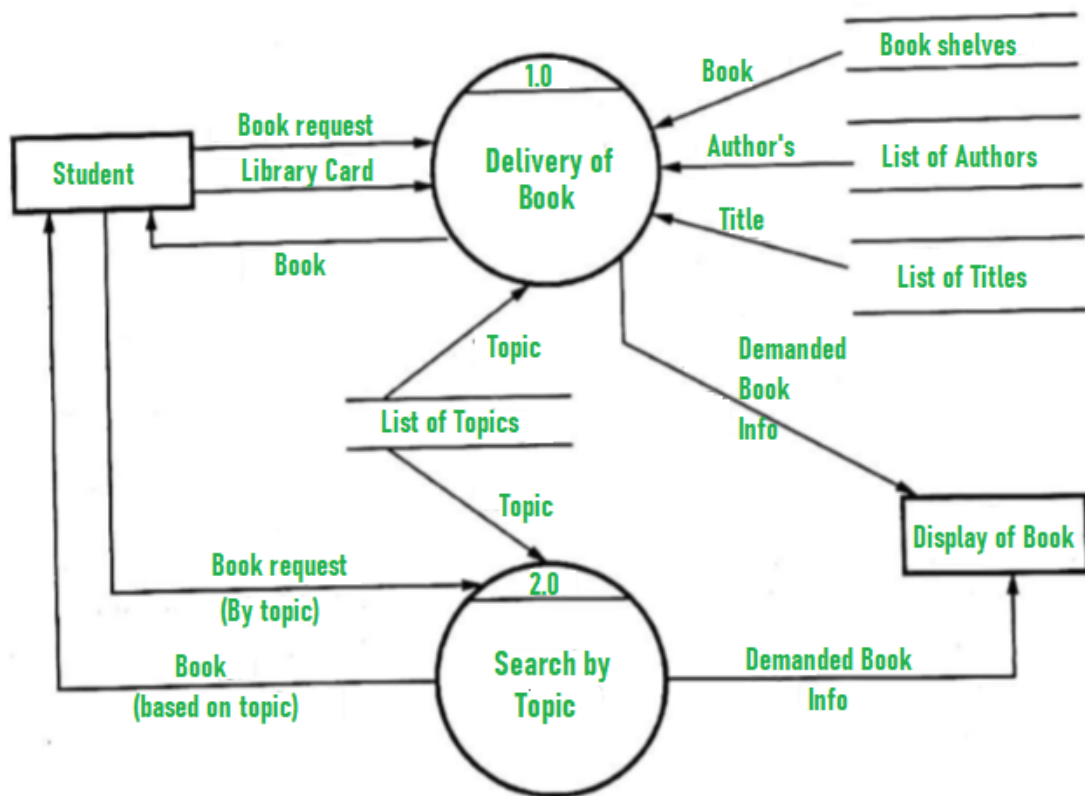
### Example of different level of DFD:

**Example: library management system:**

#### **1. Level 0(Context diagram):**

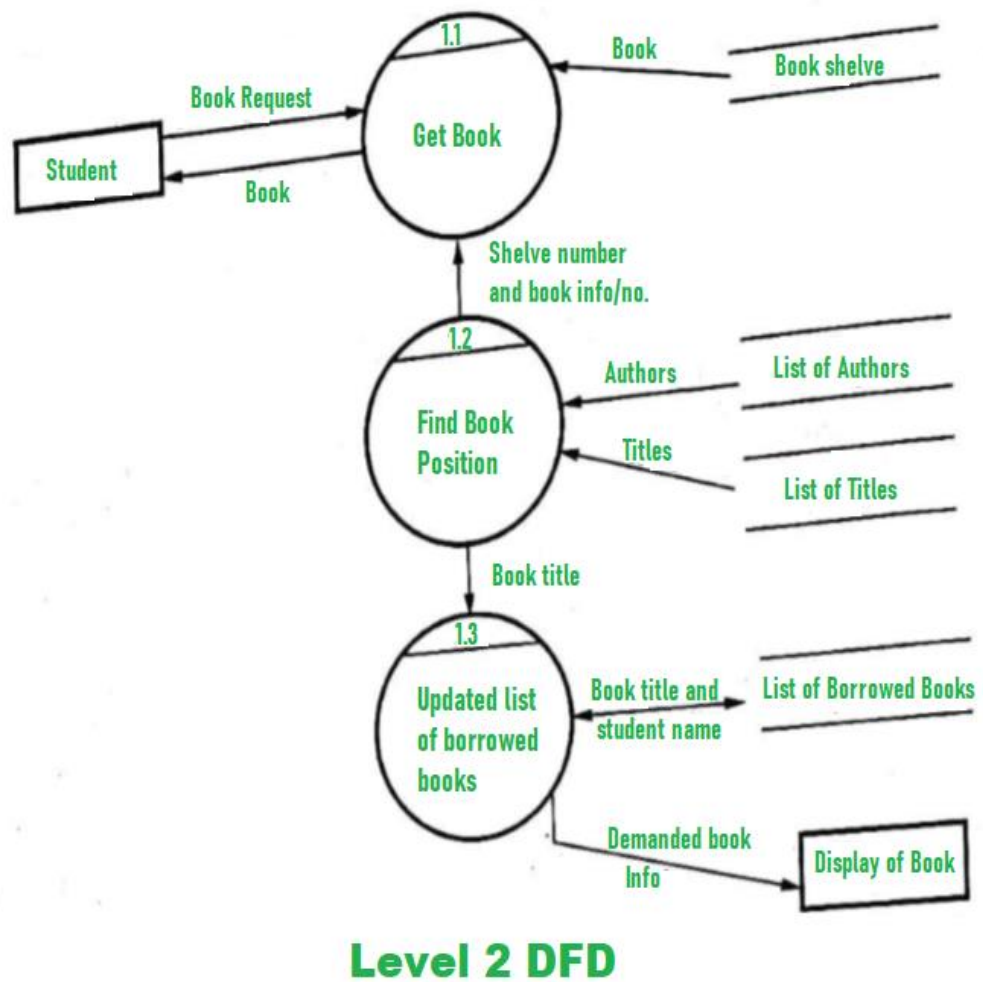


## 2. Level 1:



**Level 1 DFD**

### 3. Level 2:



### Assignments

1. What is Requirement Engineering? Explain its process.
2. Define Requirement Elicitation. Explain its technique.
3. Explain the need of use cases and user stories while performing requirement analysis.
4. Define ERD and its importance.
5. Define use case. How does use case help to understand system requirements?

6. How does DFD help to acquire functional aspect of any system? Different between the DFD level 0 and level 1.
7. What is analysis modelling? Explain its types.
8. Short notes on
  - Data dictionary
  - Software prototyping
9. Software specification vs Software specification review.
10. Draw ERD and use case of following system.
  - Hospital management system
  - Bank Atm system
  - library management system
  - Online banking system
  - Online shopping system
  - Student Registration System
  - Online movie ticket
  - School management system